

Hybrid Autonomous Navigation using Rapid Exploring Random Trees and Reinforcement Learning for Obstacle Avoidance

Balasani Venkata Ramudu
Post-Doctoral Research Scholar
School of Computing and
Informatics, University of Louisiana
Lafayette, USA
bvramu@gmail.com

Dr. Martin Margala
Director
School of Computing and
Informatics, University of Louisiana
Lafayette, USA
martin.margala@louisiana.edu

Dr. Amjan Shaik
Professor, Dept. of CSE
St Peter's Engineering College,
Maisammaguda, Hyderabad,
Telangana, India
amjanshk@gmail.com

Dr. Prasun Chakrabarti
Director, Dept. of CSE
Directorate of Research and
Publications, Dean International
Affairs, Sir Padampat Singhanian
University, Udaipur, Rajasthan, India
drprasun.cse@gmail.com

Abstract— The planning of paths is a significant issue for applications in a variety of fields, including robotics and video games. For autonomous vehicles to be implemented in practice, navigation systems are essential. Although both traditional and contemporary control techniques have been investigated, Simultaneous Localization and Mapping (SLAM) has constraints brought on by its intricacy and sensor needs. Consequently, the use of Artificial Intelligence (AI) algorithms is becoming more and more popular. This study suggests combining Rapid Exploring Random Trees (RERT) for global path planning with an algorithm for Reinforcement Learning (RL) agent acting as a local planner for avoiding obstacles. In contrast to other techniques that just use point of interest or reinforcement learning for universal path planning, this method combines random trees with reinforcement learning to guarantee effective navigation in challenging environments. The suggested approach makes navigation more effective by mapping the surroundings and creating ideal global routes that are split up into intermediate routes, toward the objective. Results from experiments show that the method works well, especially when navigating through complex landscapes, and it offers encouraging developments for autonomous navigation systems.

Keywords— *Reinforcement Learning, Robot Navigation, Path Planning, Autonomous Vehicles, Localization.*

I. INTRODUCTION

Unlike stationary robots, mobile robots can navigate in a variety of surroundings on their own, giving them adaptability. Unlike directed robots that follow predetermined courses, Autonomous Mobile Robots (AMR) function without physical or electromechanical guidance. With innovations like Industry 4.0 increasing industrial flexibility and productivity, robust navigation systems are essential for applications in homes, businesses, and healthcare cities. The field encompasses

technology that relies on sophisticated cognitive systems, including drones, unmanned rovers, and humanoid robots. AI, autonomous driving, and human-robot interaction are important themes. Robots can be classified based on their mode of mobility, including stationary, land-based, air-based, and water-based. While system-attic studies discuss modular systems, robustness, and communication issues, reviews emphasize the terrain navigation and teamwork capabilities of land-based robots. Significant recent developments with RL have also improved the navigation and control of mobile robots. Using Points of Interest (POI) driven by objectives exploration, work uses Deep Reinforcement Learning (DRL) for autonomous path planning and exploration in unknown areas. This enhances navigation efficiency and real-time mapping without the need for pre-existing maps. DRL used POI to drive a robot, but it had trouble with visible path optimization in challenging situations. The studies in this article demonstrate that the robot frequently becomes trapped in local minima due to more complicated requirements or goals concealed behind walls. By utilizing waypoints, extending POI, and keeping the robot from becoming stuck behind barriers, our approach enhances pathfinding. The number of restrictions and the goal's location determine how complex the environment is; hidden goals are particularly difficult to accomplish early on because of previous setbacks. This research explores algorithm-based learning techniques to optimize path planning in reinforcement-driven mobile robot navigation systems, enhancing efficiency and adaptability in dynamic environments. By integrating intelligent learning mechanisms, the system can autonomously navigate complex terrains while improving decision-making over time. The following sections will address the following topics: results and discussion, the environment of simulation setup (Section IV), mathematical formulation (Section III), Quantitative Analysis (Section V), and conclusion (Section VI).

II. RELATED WORKS

Many techniques exist for route planning, including the Gradient Boost Method (GBM), Graph Neural Network (GNN), Convolutional Neural Networks (CNNs), Probabilistic Roadmaps (PRM), Local Route Planning (LRP), Monte Carlo Tree Search (MCTS), and Dijkstra's and Dynamic Window Approaches (DDWAs). Improved control over robotic route planning is the goal of researchers using a range of computational methods. New studies on this subject have improved robotics via a variety of practical and theoretical improvements. The LightGBM algorithm showed promise for optimizing robotic decision-making in a credit evaluation study by Li et al. [1]. In their study, Zhang et al. [2] aimed to improve the operational accuracy and response times of robotic systems by using hardware-based real-time workload forensics. Utilizing a GNN approach that may be used for robotics network-based route planning, Liu et al. [3] used adaptive cost volume representation in stereo matching to detect rumors. Improving robot visual navigation is crucial for challenging situations, and Tong et al. [4] achieved just that. Large language models and MCTS algorithms are essential for complicated robotics decision-making frameworks; Su et al. [5] and Zhang et al. [6] addressed the strategic implications of these algorithms. For better data privacy and model robustness in networked robotic systems, Huang et al. [7], [8] investigated federated learning approaches. Ru et al. [9] provided insights and methods for trajectory planning in challenging aquatic environments. For mobility robots like unicycles, Li et al. [10] came up with a fresh strategy for planning that prioritizes tracking trajectories. The use of symmetric contrastive learning to identify defects in traffic sensors was suggested by Huang and Yang [11]. Using this technology for robotic system diagnostics guarantees dependability. An important area of robotics study, path planning determines the most secure way for the robot to go from its origin to its final destination. It may think of global and local route planning as the two primary methods. According to [12], the former gives the access to all the latest environmental data. When calculating their routes, most robots can only see what is immediately around them [13]. They have no idea what is going on in the world outside their field of vision. By design, global route planning algorithms are privy to every piece of environmental data. LRP is utilized in directions comparable to autonomous navigation, whereas global pathway planning is often used in closed, stated settings [14]. In engineering, this is often used [15]. Robots that could avoid obstacles were the first to be controlled by DRL [16]. When building a route planner, CNNs were taught to use the results of the DDWAs. Developed a map-free indoor navigation system using sparse laser radar data and DRL algorithms, and introduced a deep Q-learning-based complete strategy for mobile robot obstacle avoidance. In addition, we used Auto-RL to improve navigation within buildings and reinforcement learning using PRM to let robots traverse enormous distances.

III. PROPOSED METHODOLOGY

Three essential steps make up the suggested algorithm for moving the mobile robot: first, the environment map is pre-

processed using the RERT algorithm to find the best waypoints; next, POIs are extracted to direct the robot to the next best waypoint; and finally, local guidance is provided by a reinforcement learning algorithm which is shown in Fig. 1. The neural network navigates the robot toward the global objective by calculating actions based on sensor inputs and receiving waypoints in Polar coordinates that are relevant to the robot's location and heading.

The proposed method for moving a mobile robot works in three main steps:

First, the robot uses a method called Rapidly Exploring Random Trees (RERT) to look at the map of the environment. This map shows where the robot can and cannot go. Using this information, the algorithm finds the best possible route from the robot's starting point to its goal. It picks a set of waypoints—these are like checkpoints or smaller goals that the robot will follow to reach the final destination.

Next, the robot looks around its surroundings using sensors and identifies Points of Interest (POIs). These are safe spots or open areas that can help guide the robot to the next waypoint. The robot doesn't need to know the entire map ahead of time. Instead, it uses nearby information to move forward while avoiding obstacles like walls or moving objects. The robot updates or removes these POIs if they lead to a blocked or unsafe path.

Finally, the robot uses a Reinforcement Learning (RL) algorithm to move safely and smoothly toward each waypoint. This step is like the robot's decision-making brain. It uses sensor inputs (such as laser scans or camera data) to understand the environment and calculate the best movements—like how fast to go and in which direction. These movements are based on polar coordinates, which tell the robot how far and at what angle the next waypoint is from its current position.

A. Worldwide navigation

The RERT path planning algorithm is used to process the environment map to determine the best route from the beginning to the destination for global navigation. The map is in black and white, with non-black areas marked as navigable and black lines denoting walls and obstructions, which is shown in Fig. 2. The method refines edges and, if necessary, eliminates superfluous nodes to get the best possible path. The robot's trip is guided by the waypoints created by this preprocessing stage, which is completed at the start of navigation.

B. Navigation of points of interest (POI)

The robot uses POI to navigate to each waypoint after creating them, without being aware of the environment beforehand, it avoids dead ends and both dynamic and static barriers. Two techniques for acquiring new POIs are used by the robot to recognize and store POIs from its immediate environment. The POIs are erased from memory if they are discovered to be close to an obstruction in later stages.

Two Rplidar laser sensors, each having a maximum distance of 10 meters, were installed on the robot at various heights.

With laser readings spanning a full 180° in front of the robot, both sensors were meticulously calibrated for position and angle. The procedure is as follows:

- (i) If the disparity between two consecutive beam measurements is greater than a certain limit, signifying a navigable gap, POI is added.
- (ii) If successive beam measurements yield a categorical value, signifying open space because of the optical sensors' maximum range, POI is positioned.

In locations the robot has already visited, POIs won't be retrieved using laser scans. Another feature is that if a POI is selected as a route but cannot be surpassed within a certain number of steps, it is withdrawn, and a new route is set up. The Information-bounded Distance-constrained Exploration (IDE) assessment approach is used to choose the best waypoint at the time step from the available POI. Each potential POI's suitability is assessed using the IDE approach, which is given in Eq.(1):

$$h(c_i) = \tanh\left(\frac{ed(p_t, c_t)^2}{l1 + d(c_i, g) + e_{I_t, t}}\right) \frac{l1}{l2} \quad \text{Eq. (1)}$$

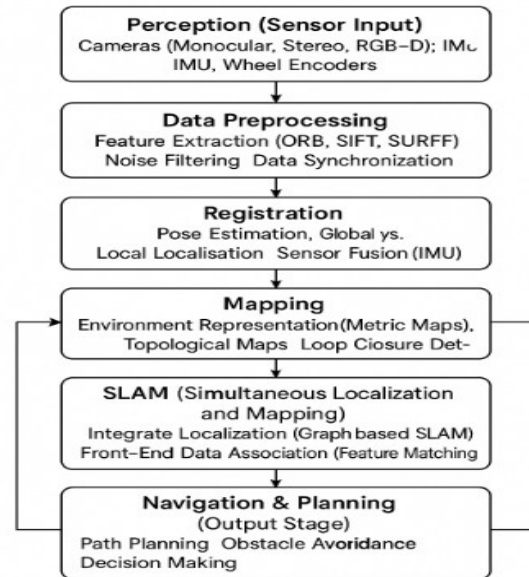
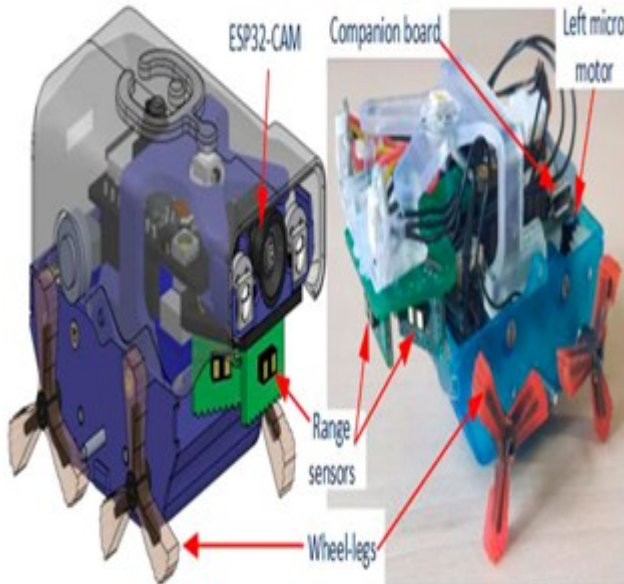


Fig. 1. The entire algorithm pipeline for the navigation of a mobile robot

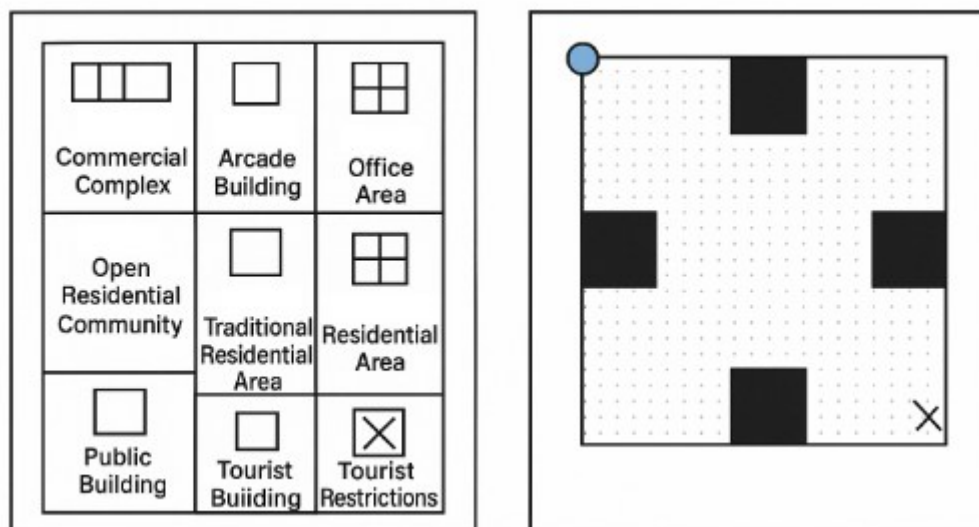


Fig. 2. (a) Building Map and (b) Boundary drawn

The sum of the three components determines the outcome h for each prospective POI , c , with coefficient i . A hyperbolic tangent $\tanh$ function given in Eq. (2) is used to describe the Euclidean measurement value $d(p_i, c_i)$ between the robot's location p at time t and the potential POI .

$$\tanh\left(\frac{ed(p_t, c_t)^2}{l_1 - l_2}\right) / 2 \quad \text{Eq. (2)}$$

Here, e stands for Euler's fractional value, while l_1 and l_2 are separation criteria that depend on the size of the settings employed in DRL training and are used to measure the score. The Euclidean distance between the potential POI and the global objective g is represented by the second component, $d(c_i, g)$. Lastly, the information score for the map at time t is given in Eq. (3) and is written as:

$$eI_t, t - (1) \quad \text{Eq. (3)}$$

Where I_i given in Eq. (4) can be calculated as:

$$I(i, t) = \frac{1}{k^2} \sum_{w=-\frac{k}{2}}^{\frac{k}{2}} \sum_{h=-\frac{k}{2}}^{\frac{k}{2}} c(x+w, y+h) \quad \text{Eq. (4)}$$

The dimensions of the kernel exploited to determine the information around the prospective points' positions x and y are denoted by k in Eq. (4), while w and h represent the top and bottom dimensions of the kernel, respectively. When navigating inside a certain area, the best POI to use is the one with the lowest IDE score, starting from 1.

C. Local Navigation

A neural network trained with the DRL algorithm and the local planner algorithm to guide the robot uses sensor input. For continuous action spaces, the motion policy is specifically trained using a Twofold Deferred Divergent and Decisive (TD3) policy gradient network. The TD3 actor network receives input from the polar coordinates of the waypoint and laser readings within an 180-degree front range. With rectified linear unit (ReLU) activation, it has two fully connected layers that output two action parameters for linear and angular velocities. These parameters are scaled by the robot's maximum velocities and confined inside $(-1, 1)$ by a $\tanh$ function. To keep the linear velocity positive, backward movement is eliminated.

$$\alpha = V_{\max} \left(\frac{\alpha + 1}{2} \right), W_{\max}, \alpha 2 \quad \text{Eq. (5)}$$

To ensure divergence, two critic networks with identical structures but delayed parameter updates evaluate the $V_{\max}$ of the corresponding pair $W_{\max}$, which is given in Eq. (5). The pair of states and actions is input to both networks. The output L_s are produced after the state is processed by a fully connected layer with ReLU activation. After being passed through two identically sized transformation fully connected (TFC) layers, this output and the action are then combined, as shown in Eq. (6)

$$L_2 = L_2 W_{r1} + \alpha W_{r2} + b_{r2} \quad \text{Eq. (6)}$$

The combined fully connected (CFC) layer, L_c , uses the weights W_{r1} and W_{r2} from layers $r1$ and $r2$, respectively, as well as a bias b from layer $r2$. After that, ReLU activation is applied to L_c , which is connected to an output layer and has a single parameter that represents the Q value. The final output is the lowest value of Q from both critic networks to avoid overestimation; the following reward function was used to compensate the agent [4].

$$r(s_t, r_t) = \begin{cases} r_g, & \text{if } Dt < \mu D \\ r_c, & \text{if collision,} \\ v - |\omega|, & \text{otherwise.} \end{cases} \quad \text{Eq. (7)}$$

At timestep t , the reward r of the state-action pair (S_t, r_t) is contingent upon three factors, which are given in Eq. (7). At the current timestep D_t , a positive goal reward r_g is provided if the distance to the target is less than the threshold μD . The detection of a collision results in the application of a negative collision reward (r_c). The present linear velocity (v) and angular velocity (ω) serve as the basis for an instantaneous reward in the absence of these conditions. To direct the navigation policy toward the objective, the following delayed ascribed reward approach is employed as given in Eq. (8):

$$r(t-i) = r(s_t - i), (\alpha - i) + r_g / i \forall i = \{1, 2, 3, 4, \dots\} \quad \text{Eq. (8)}$$

In this configuration, n is the number of previous procedures that modify the incentives. Thus, the favorable objective reward is not just assigned to the objective of the state-action pair but also dispersed (diminishingly) throughout the n steps that came before it. As a result, the network created a local navigation policy that uses direct laser input data to efficiently attain a local target while avoiding obstacles.

IV. RESULTS AND DISCUSSION

Multiple environment configurations have been used to evaluate the suggested technique, which is shown in Fig. 3.

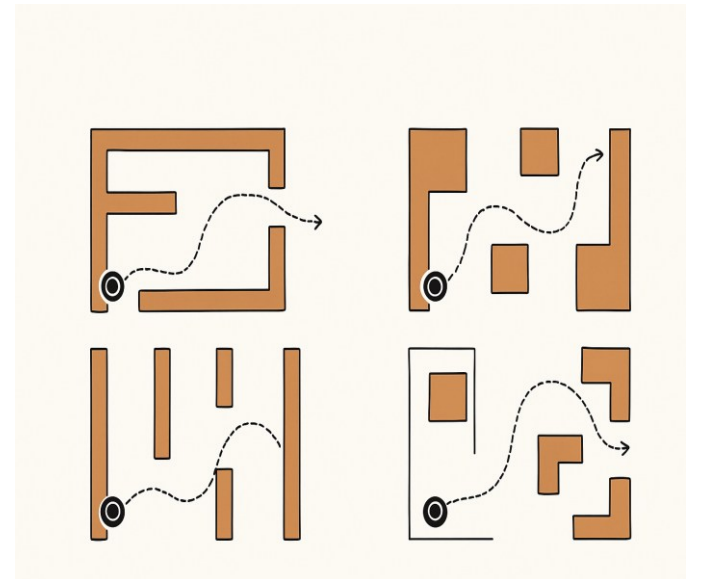


Fig. 3. Environments with varying structures and obstacle setups for testing and simulating the algorithm

A. System Setup

A PC with an Intel Core i5 CPU, 16 GB of RAM, and an NVIDIA GTX 4050 GPU was used to train local navigation using DRL. Over eight hours, the TD3 approach was trained in Gazebo using ROS commands for 800 iterations, with each episode ending when the network reached a goal, collided, or reached 500 steps. With parameter adjustments occurring every two episodes and reward delay updating over the previous ten steps, the robot's maximum speeds were 0.5 m/s and 1 rad/s. For generalization, training was conducted in a 10 x 10 m² space with noise and arbitrary obstacles. Using a Pioneer 3-DX robot equipped with RP-Lidar sensors, waypoints, and goals were deemed accomplished within a 1-meter radius. This experiment shows the viability of the concept and can be expanded to industrial-directional drive robots, albeit being smaller than actual industrial settings, which exhibits the viability of the approach and can be expanded to industrial differential drive robots.

B. Analysing quantitative data

To assess how well the algorithm guided a robot to its objective, it was tested in a variety of settings. Every configuration documented the robot's performance, contrasting the suggested method with the GDEA (Genetic Diversity Enhancement Algorithm). The GDEA is an algorithm that

enhances the diversity of the population in genetic algorithms, which is important for exploration and finding better solutions. The findings in Table I demonstrate that although the GDEA algorithm works well in straightforward settings, it has trouble in more complicated ones, frequently becoming trapped behind barriers and giving false positives for goal achievement because of local minima. On the other hand, the suggested algorithm uses waypoints to ensure consistent success in difficult contexts.

Table I measures time in seconds (*s*) to compare performance in eight environments (env-1 to env-8) and the distance *m* to the objective. The suggested algorithm's resilience is demonstrated by its consistently high performance, covering 9 m in env-1 in 31 and 7.4 m in env-8 in 17.31 s. However, the GDEA method works poorly in more complicated environments (such as env-2, env-3, and env-5) but well in simpler ones like env-1 (15s, 6.4 m) and env-4 (35s, 13.67 m). In env-7, for instance, GDEA took 96.76s as opposed to 60.63s for the suggested algorithm, demonstrating its effectiveness. The algorithms' success rates are displayed in Fig. 4. Here, the suggested algorithm obtains an 89.5% success rate, but GDEA has a 62.5% success rate, indicating limitations in complex settings, this demonstrates its resilience, dependability, and effectiveness in negotiating a variety of settings while avoiding traps such local minima that have an impact on GDEA.

TABLE I. QUANTITATIVE COMPARISON BETWEEN GDEA AND THE SUGGESTED METHOD

Algorithm	Distance and Time	ENV1	ENV2	ENV2	ENV4	ENV5	ENV6	ENV7	ENV8
Proposed algorithm	Time (s)	32	80	39.52	34.24	58.0	24.36	61.63	18.31
	Distance (m)	9	30	13.7	13.48	18.4	8.9	14	7.4
GDEA-algorithm	Time (s)	16	FAILED	FAILED	36	FAILED	12.6	96.67	12
	Distance (m)	9.6	FAILED	FAILED	13.78	FAILED	7	20	4.5

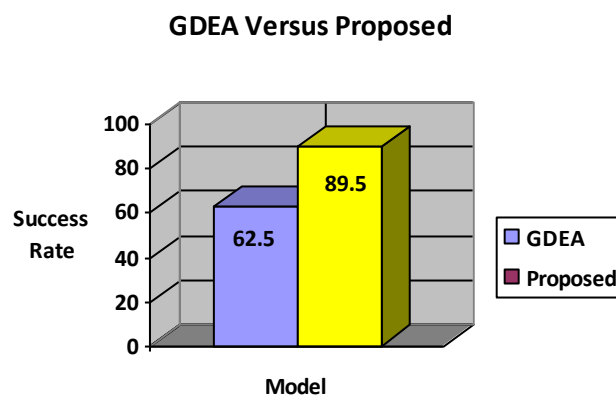


Fig. 4. Graphical Perspective: Success Rate of GDEA Versus Proposed

V. QUANTITATIVE ANALYSIS

A. Operate in a complex environment.

While the GDEA method performs poorly as complexity rises and frequently traps the robot in local minima, mistaking them for the objective, the suggested approach maneuvers the robot effectively even in complicated situations. The suggested algorithm outperforms GDEA in a variety of cases, avoiding obstacles and consistently directing the robot to its destination.

B. Robustness of the proposed algorithm

The demand for extremely accurate algorithms rises with environmental complexity, which frequently results in increased computing costs. By segmenting the intricate environment map into sub-optimal waypoints that lead the robot step-by-step to the target, the suggested technique increases navigation accuracy and robustness. By dividing difficult tasks into digestible chunks, this method guarantees dependable navigation without requiring excessive amounts of processing power and keeps the robot from becoming trapped in local minima or making serious mistakes.

C. Predictable navigation

By preprocessing an environment map to choose an optimal, predefined path, the suggested approach increases navigation accuracy and reliability. The GDEA algorithm, on the other hand, depends on judgments made in real-time without a predetermined path, which can be less dependable in complicated situations and more likely to result in local minima. The suggested algorithm guarantees more successful and consistent navigation in a variety of situations by utilizing waypoints and a prearranged route.

VI. CONCLUSION

Finally, by combining the RERT global path planning algorithm with RL, this work offers a revolutionary approach to autonomous vehicle navigation. The suggested approach creates an ideal global path from the starting point to the end goal, in contrast to earlier approaches, which depended on POI for worldwide navigation and lacked explicit optimization for pathfinding. By using RERT for global planning, the technique eliminates problems like local minima that beset previous methods and offers a clear, optimized route. To generate waypoints along the best route, the approach first analyses an image map of the surroundings to find impediments. Its RL-based local planner is guided by these waypoints to navigate efficiently and stay clear of obstructions. This method not only guarantees a more effective navigation process but also improves the robot's capacity to accomplish its objective despite complicated environmental conditions. The suggested method's drawbacks, which will be the focus of further study, are mostly associated with how it behaves in simpler environments. The suggested method might create pointless waypoints, causing the robot to travel farther and take less-than-ideal routes, even though the

GDEA algorithm might work effectively by adjusting to immediate obstacles. More effective path-planning algorithms, such as RERT*, can enhance this. However, the suggested approach keeps the robot from becoming trapped in local minima in complicated situations. The robot is more likely to succeed in achieving its objective when RERT is used for global planning and RL is used for local navigation. The difficulties that arise when implementing the technique in the intricate setting of an actual industrial setting with industrial robots are another subject that needs more research.

References

- [1] Li S, Dong X, Ma D, et al. Utilizing the LightGBM Algorithm for Operator User Credit Assessment Research. arXiv preprint arXiv:2403.14483, 2024.
- [2] Zhang Y, Zhou L, Makris Y. Hardware-based real-time workload forensics via frame-level TLB profiling. IEEE 37th VLSI Test Symposium, 2019.
- [3] Liu T, Cai Q, Xu C, et al. Rumor Detection with a Novel Graph Neural Network Approach. arXiv preprint arXiv:2403.16206, 2024.
- [4] Tong K W, Sun P Z H, Wu E Q, et al. Adaptive cost volume representation for unsupervised high-resolution stereo matching. IEEE Transactions on Intelligent Vehicles, 2022.
- [5] Su J, Jiang C, Jin X, et al. Large Language Models for Forecasting and Anomaly Detection: A Systematic Literature Review. arXiv preprint arXiv:2402.10350, 2024.
- [6] Zhang Y, Zhu M, Gui K, et al. Development and Application of a Monte Carlo Tree Search Algorithm for Simulating DaVinci Code Game Strategies. arXiv preprint arXiv:2403.10720, 2024.
- [7] Huang W, Wan G, Ye M, et al. Federated graph semantic and structural learning. Int. Joint Conf. Artif. Intell., 2023.
- [8] Wan G, Huang W, Ye M. Federated Graph Learning under Domain Shift with Generalizable Prototypes. AAAI Conference on Artificial Intelligence, 2024.
- [9] Ru J, Yu H, Liu H, et al. A Bounded Near-Bottom Cruise Trajectory Planning Algorithm for Underwater Vehicles. Journal of Marine Science and Engineering, 2022.
- [10] Li S, Zhang G, Lei X, et al. Trajectory tracking control of a unicycle-type mobile robot with a new planning algorithm. IEEE International Conference on Robotics and Biomimetics, 2017.
- [11] Huang Y, Yang J J. Symmetric contrastive learning for robust fault detection in time-series traffic sensor data. International Journal of Data Science and Analytics, 2024.
- [12] Chen Z, Jing L, Li Y, et al. Bridging the domain gap: Self-supervised 3d scene understanding with foundation models. Advances in Neural Information Processing Systems, 2024, 36.
- [13] Xie H, Deng T, Wang J, et al. Robust incremental long-term visual topological localization in changing environments. IEEE Transactions on Instrumentation and Measurement, 2022, 72: 1-14.
- [14] Jaladi S R, Chen Z, Malayanur N R, et al. End-To-End Training and Testing Gamification Framework to Learn Human Highway Driving. 2022 IEEE 25th International Conference on Intelligent Transportation Systems (ITSC). IEEE, 2022: 4296-4301.
- [15] Deng T, Xie H, Wang J, et al. Long-Term Visual Simultaneous Localization and Mapping: Using a Bayesian Persistence Filter-Based Global Map Prediction. IEEE Robotics & Automation Magazine, 2023, 30(1): 36-49.
- [16] Tai, L.; Liu, M. Towards Cognitive Exploration through Deep Reinforcement Learning for Mobile Robots. arXiv 2016, arXiv:1610.01733.